

Artificial Intelligence in Games

CA3 – Integrated AI Project + Evaluation

Brona Keevers-Roux

20098883

Repo: <https://github.com/hillyleopard133/4th-year-game-dev/tree/ca3>

Presentation: <https://youtu.be/BXB1mqlefsQ>

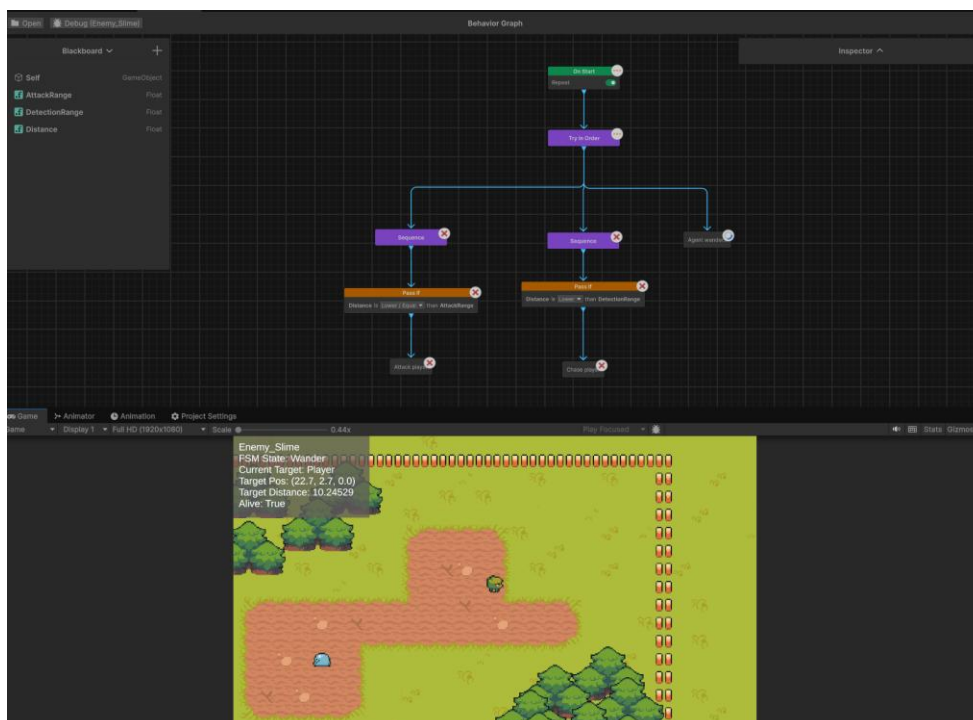
Unity Video: <https://youtu.be/XPr8yMRIV1A>

Integrated Technical Account

The AI system is built around a Behaviour Tree (BT) that controls high-level decision-making, supported by action scripts for movement and a grid-based A* navigation system for pathfinding. These systems are connected through a shared EnemyBrain component and a small set of blackboard variables that store runtime information such as distance to the player and behavioural thresholds.

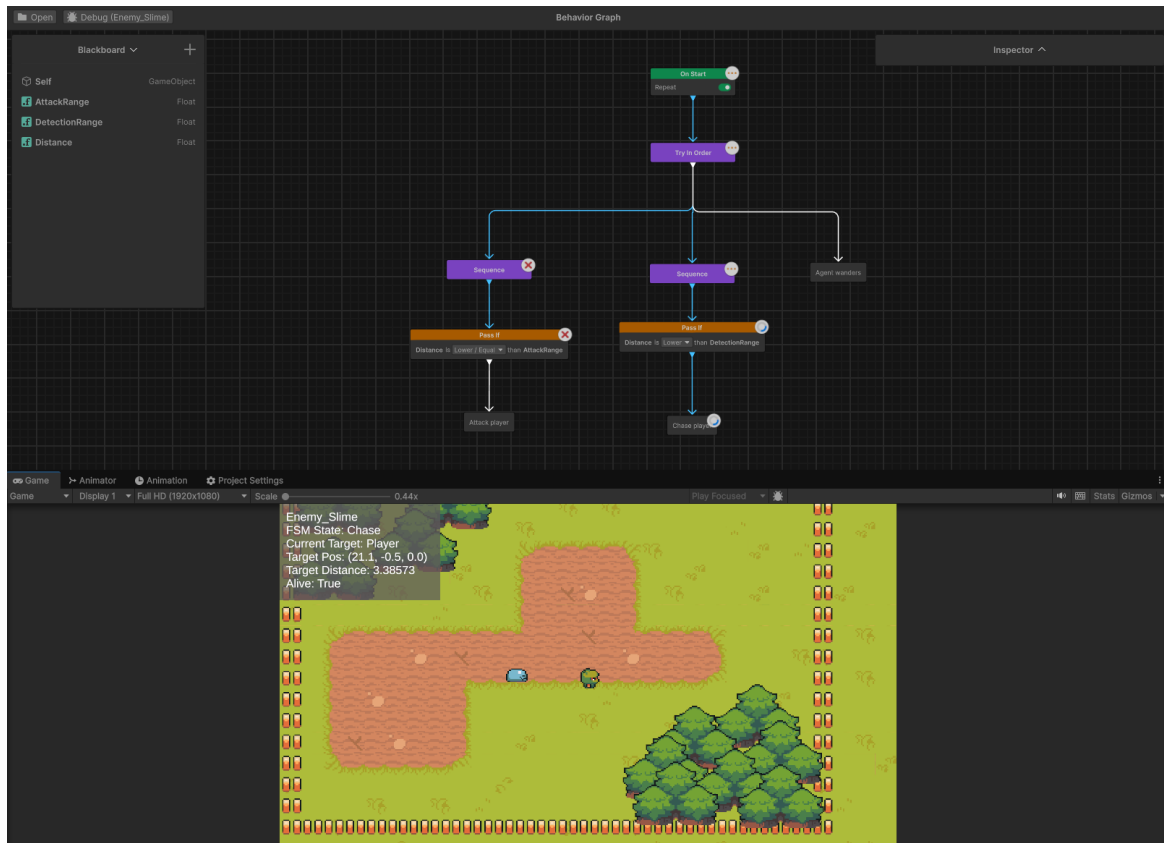
The core of the system is a single continuously updated value: the distance between the enemy and the player. This value is calculated every frame and written into the Behaviour Tree blackboard. All behaviour transitions are driven by comparisons between this distance value and predefined detection and attack ranges. This makes the system fully data-driven, with no explicit state machine transitions in code, as state changes are handled via Behaviour Tree evaluation.

Fig 1: Wander state behaviour



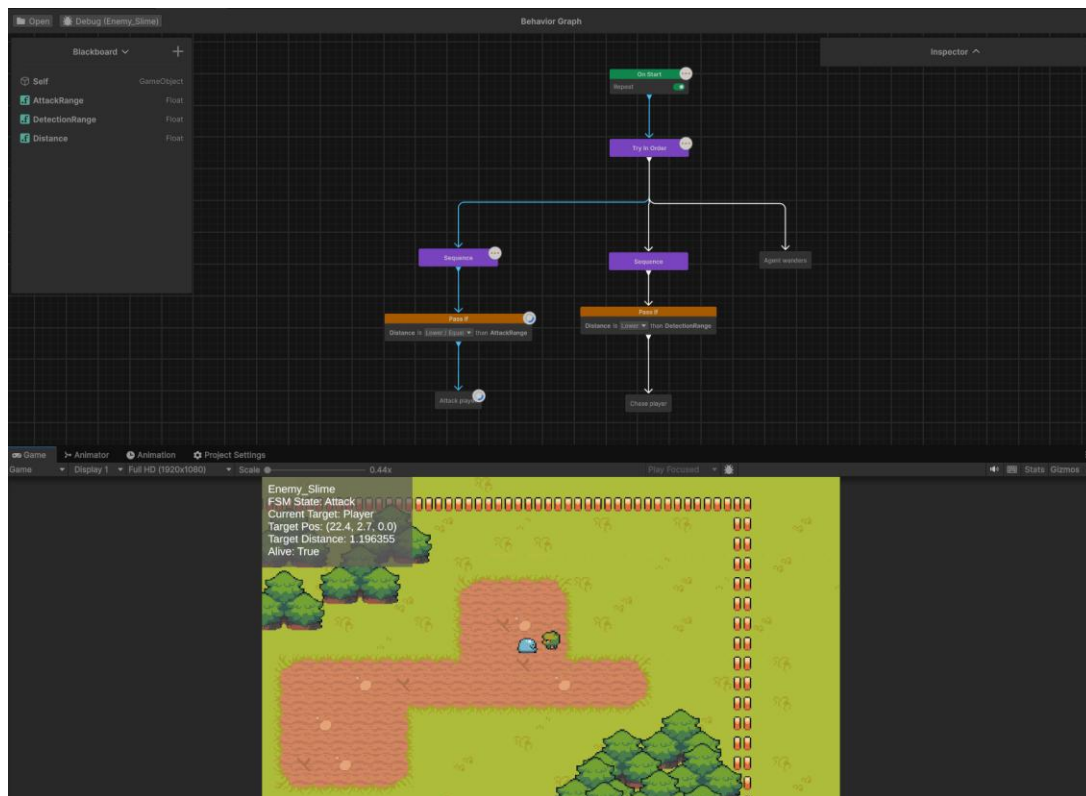
When the player is outside detection range, the Behaviour Tree defaults to Wander behaviour. In this state, the agent selects random destinations within a defined area and uses A* pathfinding to move between them. Behaviour remains lightweight as path recalculation only occurs intermittently based on timers and movement completion.

Fig 2: Chase state behaviour



When the distance value drops below the detection threshold, the Behaviour Tree transitions into Chase. In this state, the AI continuously updates its path towards the player using A* pathfinding. This makes Chase the most computationally expensive behaviour due to frequent path invalidation and recalculation driven by player movement.

Fig 3: Attack state behaviour



When the player enters attack range, the Behaviour Tree transitions into Attack. This state focuses on maintaining close proximity and applying damage when conditions are met. Unlike Chase, Attack does not use pathfinding and instead relies on timing logic and range checks to determine when damage is applied.

A key design decision was to centralise all perception into a single distance value rather than using multiple detection systems. This simplified both debugging and tuning, since all behaviour is controlled by the same underlying variable.

The EnemyBrain component acts as a shared coordination layer, holding references to movement, physics, and animation systems. This ensures that both the Behaviour Tree and action scripts operate on consistent runtime data without direct coupling.

During development, the Behaviour Tree was introduced as the primary decision-making system, replacing most earlier FSM-style logic. The remaining FSM-style scripts were refactored into execution-only components responsible for movement and animation rather than state transitions. This improved clarity between decision-making and behaviour execution.

Debug overlays and gizmos were used to validate system behaviour. These were especially important for tuning transition thresholds, ensuring smooth switching between Wander, Chase, and Attack without oscillation or incorrect range detection.

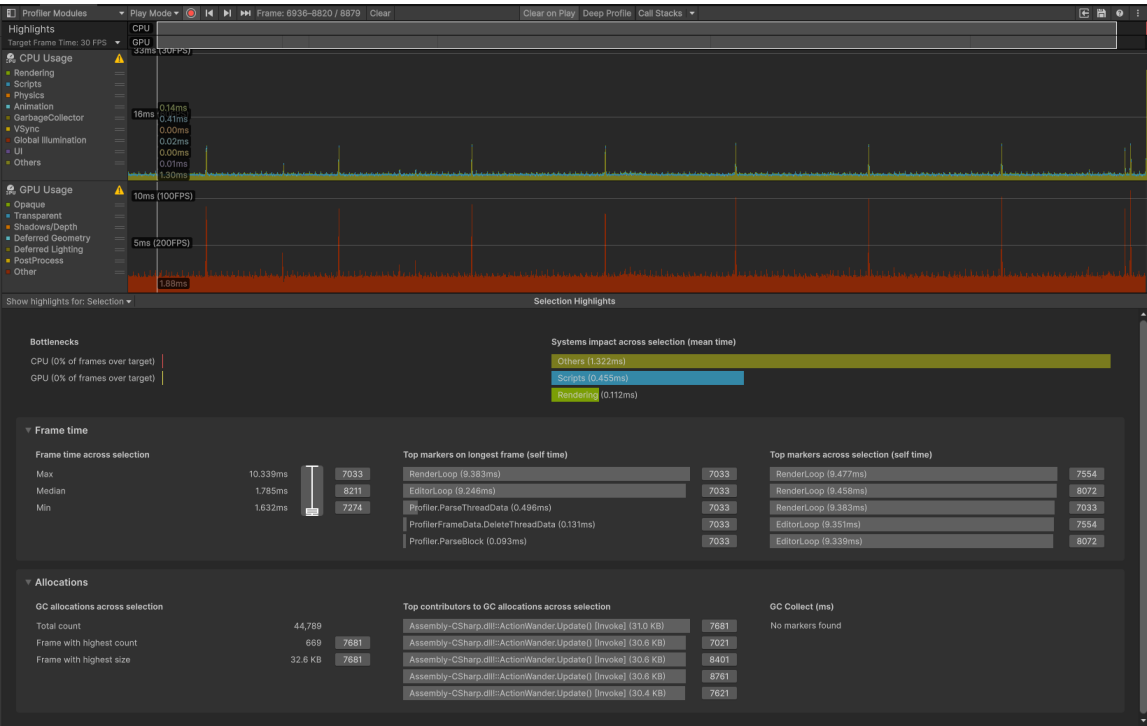
Overall, the system follows a clear structure: a single distance-based perception value drives Behaviour Tree decisions, which select behaviours executed by modular action scripts, all supported by a shared navigation system. This results in a predictable and easily debugged AI pipeline.

Performance and Robustness Evaluation

Profiling was conducted using Unity's Profiler in Play Mode across three AI states: Wander, Chase, and Attack. The aim was to evaluate how computational cost varies between behaviours and identify the main performance bottlenecks within the AI system.

The results show a clear cost hierarchy. The Chase state is the most expensive, followed by Attack, with Wander being the least expensive overall. Behaviour Tree logic remains lightweight in all cases, with performance differences primarily driven by navigation and the frequency of system updates.

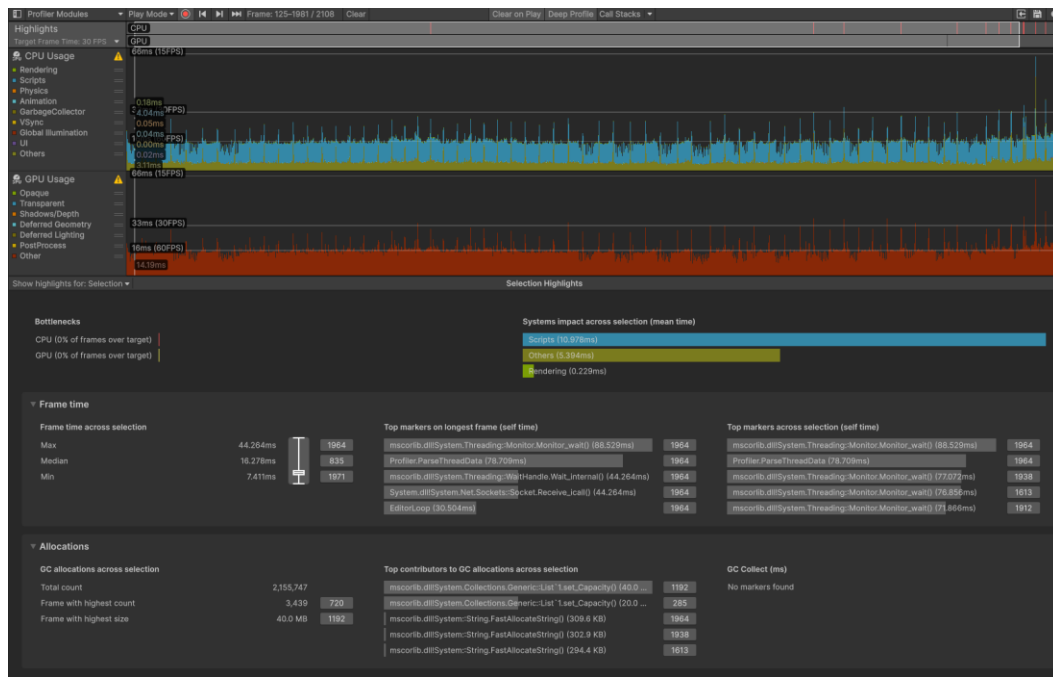
Fig 4: Wander state profiling



This shows the lowest CPU usage profile, where behaviour is primarily driven by intermittent updates and minimal pathfinding. Most processing occurs only when new destinations are generated, resulting in a relatively flat performance profile compared to other states.

The main computational cost comes from A* pathfinding during Chase. When the player is detected, the AI repeatedly generates paths using a grid-based search (AStar.BuildPath). This becomes expensive due to frequent recalculation caused by player movement and path invalidation. In the Profiler, this appears as CPU spikes under Scripts during active pursuit, confirming navigation as the primary bottleneck.

Fig 5: Chase state profiling

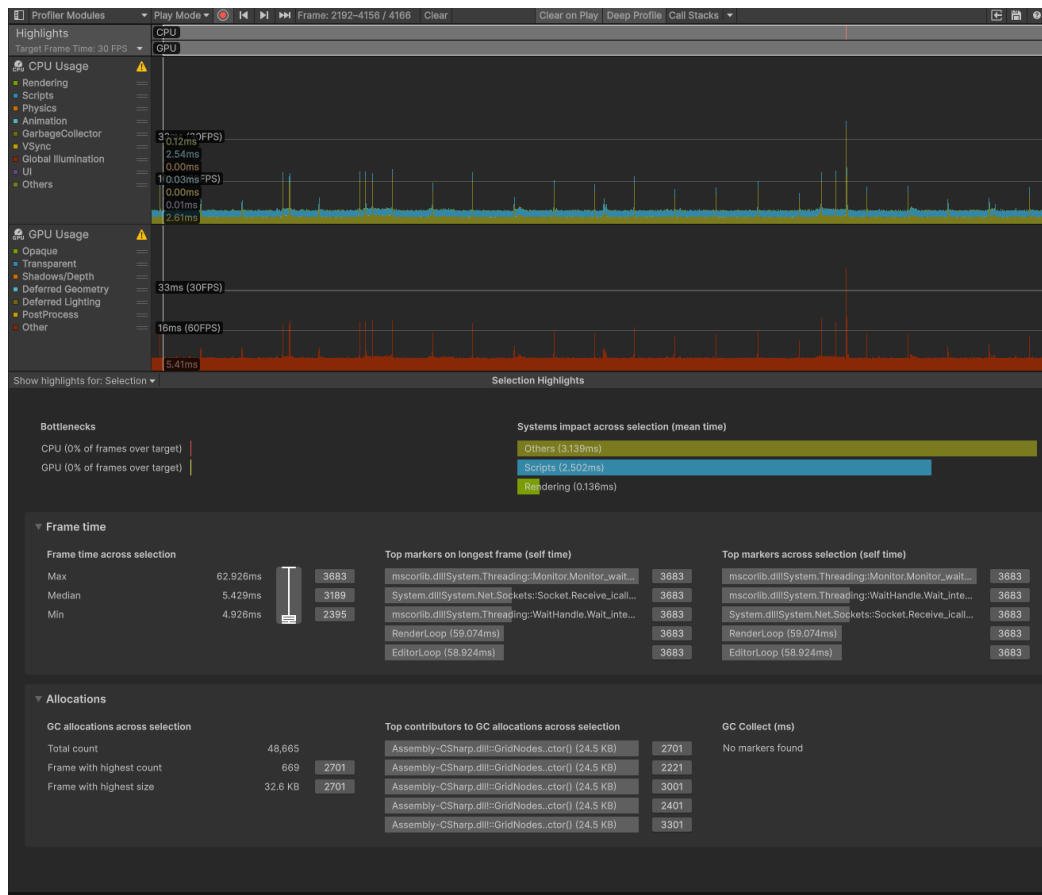


This highlights the highest CPU usage, with noticeable spikes corresponding to path recalculation. The cost is driven by repeated A* execution and frequent updates to movement targets as the player moves.

To address this, pathfinding is throttled using a cooldown system to prevent continuous recalculation every frame. A frame-spreading technique is also used to distribute pathfinding across multiple frames, reducing peak CPU load when multiple agents are active. In addition, a distance threshold ensures paths are only recalculated when the player has moved significantly, reducing unnecessary updates. Together, these measures reduce spikes while maintaining responsive chasing behaviour.

Attack performance also appears as a spiky profile in the Profiler, but with noticeably higher spike frequency than Wander. Although it does not involve navigation or pathfinding, it executes continuous per-frame logic including timer updates, conditional checks, animation state changes, and periodic damage application. These repeated evaluations produce frequent small bursts of activity, particularly around attack timing intervals and state transitions.

Fig 6: Attack state profiling



This shows repeated CPU spikes occurring more regularly than in Wander. The cost is driven by sustained combat logic rather than occasional path generation, resulting in a more consistently active profile even though individual spikes are smaller than those seen in Chase.

Wander, by contrast, produces fewer but more isolated spikes. These are primarily caused by pathfinding operations when new destinations are generated. Between these spikes, execution remains minimal due to early exits in the update loop, resulting in a lower overall frequency of activity.

Two key edge cases were encountered. The first involved null player references during respawn or transitions, which previously caused invalid pathfinding calls. This was resolved with defensive null checks before movement logic. The second involved enemies stopping slightly outside attack range due to waypoint tolerance and grid resolution, preventing consistent state transitions into Attack. This was fixed by tightening movement thresholds and separating attack range from navigation stopping distance.

Overall, the system remains stable and predictable. The Behaviour Tree and decision layers are lightweight, while navigation remains the dominant cost factor in Chase. Performance is primarily determined by the frequency of expensive operations rather than their individual complexity, and the implemented throttling and scheduling strategies keep runtime cost under control.

AI in Modern Engines and Beyond

Modern game engines provide extensive AI tooling, but Unity 6 and Unreal Engine 5 differ significantly in architectural philosophy, particularly in how AI systems are structured, integrated, and extended. These differences affect both gameplay development and simulation-oriented applications such as training environments.

Unity 6 follows a modular, code-driven architecture where AI systems are composed from discrete components such as scripts and navigation systems. The Unity AI Navigation package provides NavMesh-based pathfinding, enabling agents to compute and follow traversable paths across game worlds. However, higher-level AI systems such as Behaviour Trees, blackboards, and perception pipelines are not fully integrated into the engine and must be implemented manually or through extensions. This results in a flexible but low-level architecture where system integration is explicitly controlled by the developer .

Unreal Engine 5 provides a more integrated AI framework built around Behaviour Trees, Blackboard data structures, and AI Perception. Behaviour Trees structure decision-making into hierarchical nodes such as selectors, sequences, and tasks, allowing clear separation between decision logic and execution. The Blackboard acts as shared runtime memory, enabling different parts of the AI system to access and update common state data without direct coupling. This produces a structured decision pipeline that improves scalability and maintainability .

Unreal's AI Perception system extends this architecture by allowing agents to detect environmental stimuli such as sight and hearing. These stimuli automatically update Blackboard values, enabling Behaviour Trees to react dynamically without manual polling or custom state synchronisation. This creates a consistent perception -> decision -> action pipeline widely used in both gameplay AI and simulation systems .

Despite these differences, both engines rely on shared foundational AI principles: navigation systems for movement, hierarchical decision structures for behaviour control, and shared runtime state for coordination. The primary distinction lies in abstraction level rather than capability.

Beyond entertainment games, these AI architectures are also widely used in serious games and training simulation environments, where AI behaviour must be structured, repeatable, and controllable. Behaviour Tree-based systems are particularly suited to these environments because they provide explicit, inspectable decision logic and predictable execution flow. This makes them appropriate for scenarios where AI agents represent trainees, adversaries, or controlled environmental actors in simulation-based training systems.

A clear example of this approach can be seen in research on Behaviour Tree-based computer-generated forces (CGF) used in simulation and training environments. In these systems, Behaviour Trees are used to model structured tactical behaviour for simulated entities, allowing scenarios to be replayed and evaluated consistently. This is important in training contexts where repeatability and behavioural consistency are required for assessment rather than entertainment.

Similarly, Behaviour Trees are widely discussed in the context of serious games for training and education, where AI systems are designed to support learning outcomes rather than entertainment. Research on serious game AI highlights the use of structured decision systems, adaptive agents, and controlled behaviour models to ensure that training scenarios remain consistent while still responsive to user interaction .

This introduces a key distinction between serious games AI and commercial game AI. In commercial games, AI systems are designed to create engaging and dynamic behaviour through variability, unpredictability, and adaptive challenge. In contrast, serious games and simulation AI prioritise determinism, auditability, and repeatability. Behaviour must be explainable and consistent so that trainee performance can be measured against stable conditions.

These requirements introduce additional constraints such as reduced randomness, structured state transitions, and predictable decision pathways. AI systems must remain within defined behavioural boundaries, and their logic must be traceable for evaluation purposes. As a result, Behaviour Trees and blackboard-based architectures are commonly used in both game engines and simulation environments due to their clarity and controllability.

Overall, Unity 6 and Unreal Engine 5 represent two different points on the AI design spectrum. Unity prioritises flexibility and low-level control through modular components, while Unreal provides a highly integrated AI framework built around Behaviour Trees, Blackboard systems, and perception pipelines. When extended into serious games and training simulation contexts, these systems demonstrate how game AI techniques generalise into broader autonomous system design, where structure, interpretability, and repeatability become more important than emergent gameplay behaviour.

References

Unity Technologies (2025) *AI Navigation Package Documentation*

<https://docs.unity3d.com/Packages/com.unity.ai.navigation@2.0/manual/index.html>

Epic Games (2025) *Behavior Trees in Unreal Engine*

<https://dev.epicgames.com/documentation/unreal-engine/behavior-trees-in-unreal-engine>

Epic Games (2025) *AI Perception in Unreal Engine*

<https://dev.epicgames.com/documentation/en-us/unreal-engine/ai-perception-in-unreal-engine>

Westera, W. et al. (2019) *Artificial intelligence moving serious gaming: Presenting reusable game AI components*

<https://link.springer.com/article/10.1007/s10639-019-09968-2>

Carry-forward and professional reflection

If I continued working on this project, the main improvement I would make is separating perception from decision making more cleanly. At the moment, I rely mainly on a single distance-to-player value stored in the Blackboard to drive both state changes and behaviour. A more structured approach would be to introduce a dedicated perception layer (for example sight checks, awareness levels, or a simple suspicion system), allowing the Behaviour Tree to operate on richer state information rather than raw distance alone.

One of the key insights from reviewing Unreal Engine's Behaviour Tree and AI Perception systems, alongside the wider discussion around serious games, is the importance of AI readability. It is not only about whether the AI behaves correctly in game, but whether its decisions can be understood and traced. In training or simulation contexts, this becomes especially important, as behaviour needs to be consistent and explainable rather than just convincing.

In terms of my own development, the main learning point was how sensitive AI behaviour becomes when multiple systems interact in real time. Even though the FSM, Behaviour Tree, and navigation systems worked together, most of the debugging effort focused on small issues such as Blackboard update timing, threshold values, and recalculation frequency. I also encountered cases where the AI was technically in the correct state but behaved incorrectly in practice, such as stopping just outside attack range and failing to transition cleanly between chase and attack.

This highlighted that AI systems are less about isolated components functioning correctly and more about how timing, thresholds, and shared data interact during runtime. Small design decisions, such as where values are calculated or how strictly conditions are evaluated in the Behaviour Tree, had a larger impact on behaviour than expected.

Overall, the project shifted my focus from thinking about AI as separate behaviours that operate independently, to thinking about runtime behaviour stability. The main concern became not whether individual systems work, but whether the combined system behaves consistently and predictably under gameplay conditions.